Lab3db

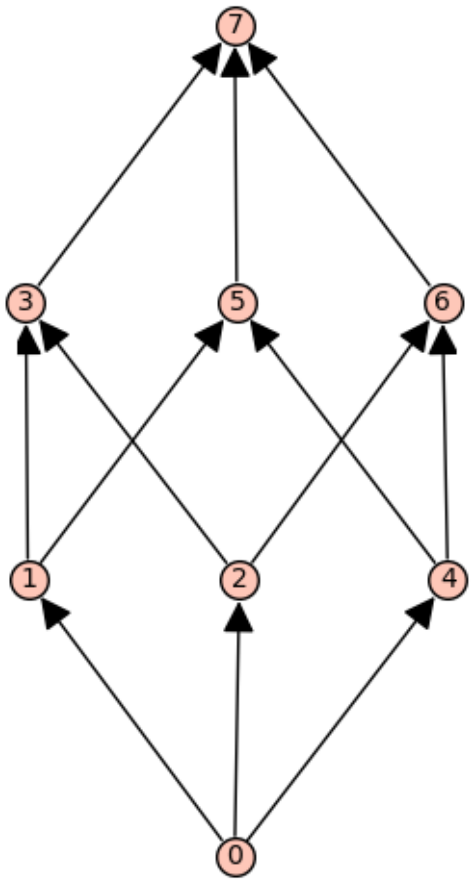
May 14, 2026

1 Mobius Functions, Order Polynomials, and Linear Extensions

1.1 Math 737 - Lab 3

1.2 Sage can compute the Mobius function of a poset. Create a poset and then use the commands moebius_function and moebius_function_matrix to determine how they work and how they relate to Mobius function computations done by hand.

[29]:
```
B = Posets.BooleanLattice(3)

show(B.plot())

print(B.moebius_function(0,7))

B.moebius_function_matrix()
```




```
-1
```

[29]:
```
[ 1 -1 -1  1 -1  1  1 -1]
[ 0  1  0 -1  0 -1  0  1]
[ 0  0  1 -1  0  0 -1  1]
[ 0  0  0  1  0  0  0 -1]
[ 0  0  0  0  1 -1 -1  1]
[ 0  0  0  0  0  1  0 -1]
[ 0  0  0  0  0  0  1 -1]
[ 0  0  0  0  0  0  0  1]
```

1.3 Exercise 1: Use the above commands to compute the Mobius function of some of our example posets P, Q, R, etc. Now compute the Mobius function of the product of these posets. What fact about Mobius functions does this show you?

Example Poset Constructions

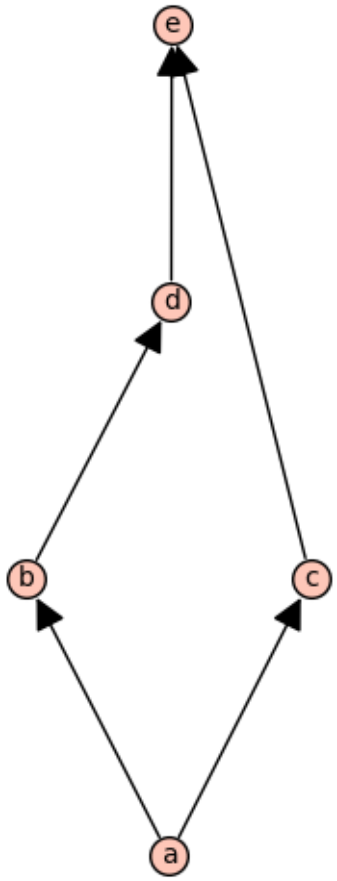
[56]:
```
# Example Poset Constructions

# P
elem = ['a','b','c','d','e']
rels = [['a','b'],['b','d'],['d','e'],['a','c'],['c','e']]
P = Poset((elem,rels))
P.plot()

# Q
elem = ['a','b','c','d','e']
rels = [['a','b'],['a','c'],['b','d'],['c','d'],['c','e'],['d','f'],['e','f']]
Q = Poset((elem,rels))
Q.plot()

# R
elem = ['a','b','c']
rels = [['a','c'],['b','c']]
R = Poset((elem,rels))
R.plot()

# S
elem = ['a','b','c','d']
rels = [['a','c'],['a','d'],['b','c'],['b','d']]
S = Poset((elem,rels))
S.plot()

# T
elem = ['a','b','c','d']
rels = [['a','b'],['b','d'],['a','c']]
T = Poset((elem,rels))
T.plot()

# N
elem = ['a','b','c','d']
rels = [['a','c'],['b','c'],['b','d']]
N = Poset((elem,rels))
N.plot()

# Poset dictionary
PO = {
    "P": P,
    "Q": Q,
    "R": R,
    "S": S,
    "T": T,
    "N": N
}
```

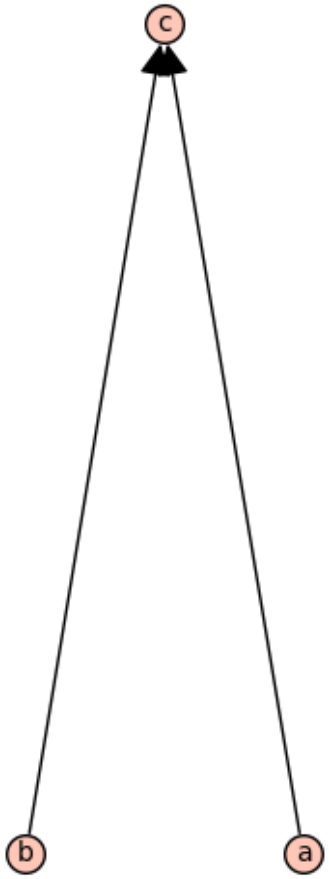
```
"""
examples of poset operations:

# Disjoint sum (disjoint union) of T and R
PdQ = Q.disjoint_union(P)
# Ordinal sum of P and Q
PoQ = P.ordinal_sum(Q)
# Cartesion product of P and Q
PxQ = P.product(Q)
# Ordinal Product of P and Q
PoxQ = P.ordinal_product(Q)
"""

RdT = R.disjoint_union(T)
RoN = R.ordinal_sum(N)
NxR = N.product(R)
NoxR = N.ordinal_product(R)

ProdPO = {
    "RdT": RdT,
    "RoN": RoN,
    "NXR": NxR,
    "NoxR": NoxR,
}
```

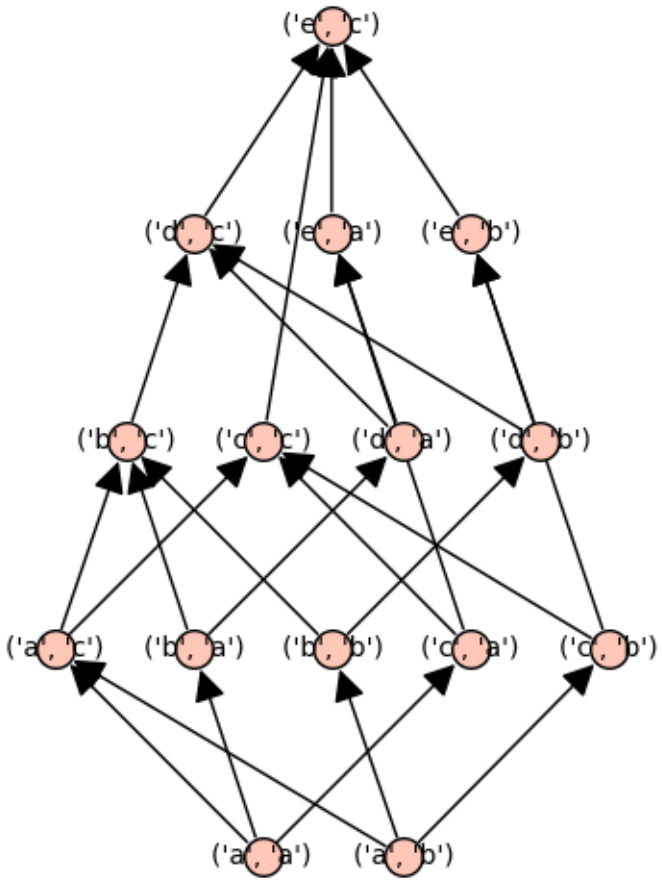
Mobius computations: An example with 0, some examples of interval computations from and example poset without 0, and then some product computations

[59]:
```
# Show the posets
show(P.plot())
show(R.plot())

PxR = P.product(R)

show(PxR.plot())

# P has a bottom element
print("P has bottom:", P.bottom())

print("\nP:")
print("mu(a,b) =  mu(b) =", P.moebius_function('a','b'))
print("mu(a,d) = mu(d) =", P.moebius_function('a','d'))
print("mu(a,e) = mu(e) =", P.moebius_function('a','e'))


# R does not have a bottom element
```

```
print("\nR has no 0:")

print("\nR:")
print("mu(a,c) =", R.moebius_function('a','c'))
print("mu(b,c) =", R.moebius_function('b','c'))


# Product poset P x R
PxR = P.product(R)

print("\nP x R:")

print(
    "mu((a,a),(e,c)) =",
    PxR.moebius_function(('a','a'), ('e','c'))
)

print(
    "mu_P(a,e) * mu_R(a,c) =",
    P.moebius_function('a','e') * R.moebius_function('a','c')
)
```




c
b
a




```
P has bottom: a

P:
mu(a,b) =  mu(b) = -1
mu(a,d) = mu(d) = 0
mu(a,e) = mu(e) = 1

R has no 0:

R:
mu(a,c) = -1
mu(b,c) = -1

P x R:
mu((a,a),(e,c)) = -1
mu_P(a,e) * mu_R(a,c) = -1
```

We can see that the Möbius function of a product poset is equal to the product of the mobius function of all of it's component posets. That is:

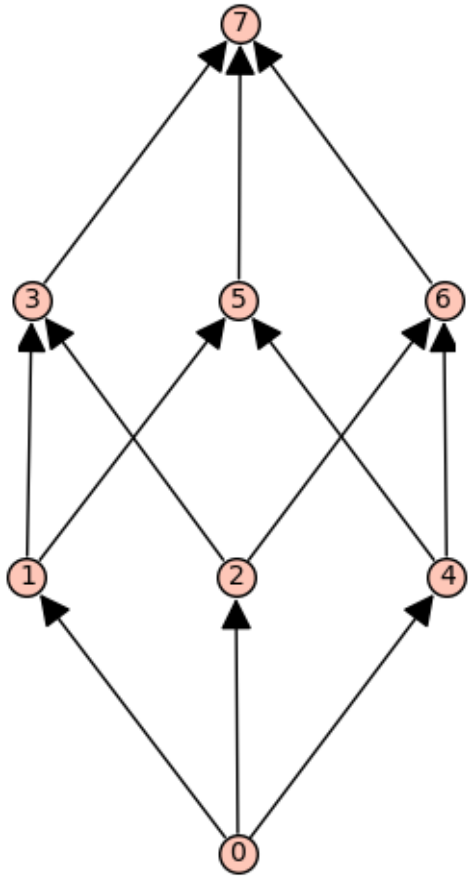
mu_{P x R}((a,a),(e,c)) = mu_P(a,e) mu_R(a,c).

More generally,

mu_{P x Q}((p,q),(p',q')) = mu_P(p,p') mu_Q(q,q').

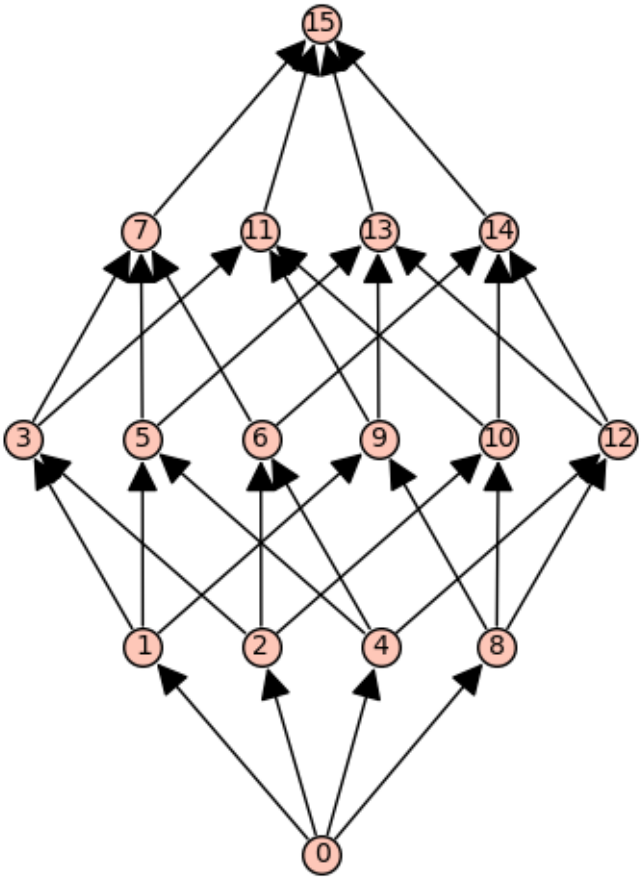
1.4 Exercise 2: Compute the Mobius function of the Boolean lattice for n=3,4. What do you notice? Do the same for the strong Bruhat order.

[68]:
```
B3 = Posets.BooleanLattice(3)
show(P3.plot())
B3.moebius_function(0,7)
```



[68]: -1

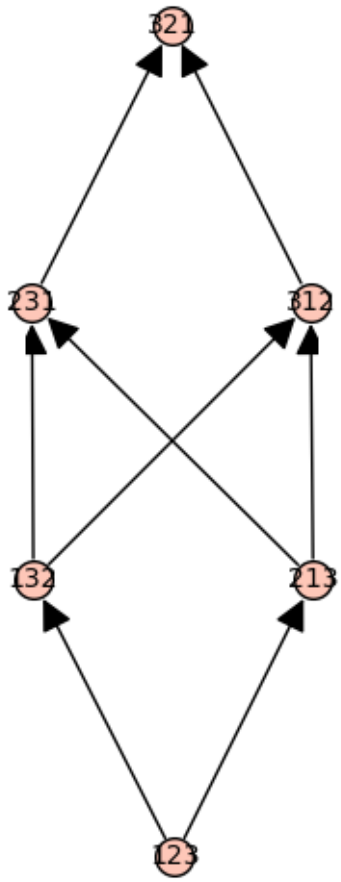
[61]:
```
B4 = Posets.BooleanLattice(4)
show(B4.plot())
B4.moebius_function(0,15)
```



[61]: 1

For BooleanLattice(n), the mobius function is mu(S)= -1^{|S|}

[65]:
```
SymBO= Posets.SymmetricGroupBruhatOrderPoset(3)
show(SymBO.plot())
SymBO.moebius_function('123','321')
```



[65]: -1

For SymmetricGroupBruhatOrderPoset(3) the mobius function is mu(length(sigma)) where length is the minimum number of transpositions whose product equals sigma

1.5 Exercise 3: Compute the order polynomial and the number of linear extensions of the following posets:

1.5.1 - An antichain of 5 elements

1.5.2 - A chain of 5 elements

1.5.3 - A poset that is the product of two chains of 3 and 5 elements

To find the number of linear extensions, you may find it helpful to first find the command to generate all linear extensions and then put the whole statement inside the command len() which finds the length of this list.

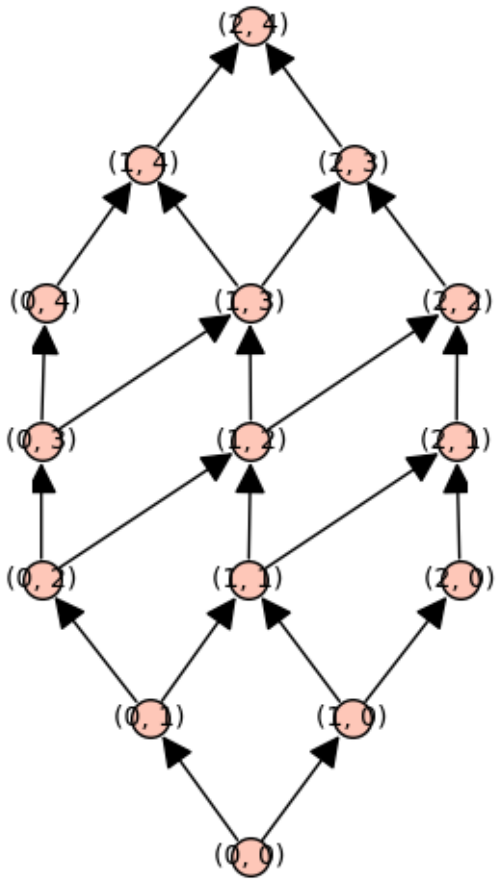
[72]:
```
# Exercise 3

A5 = Posets.AntichainPoset(5)
```

```
C5 = Posets.ChainPoset(5)
C3 = Posets.ChainPoset(3)

C3xC5 = C3.product(C5)

posets = {
    "Antichain of 5 elements": A5,
    "Chain of 5 elements": C5,
    "Product of chains C3 x C5": C3xC5
}

for name, P in posets.items():
    print("\n" + name)
    show(P.plot())

    print("Order polynomial:")
    print(P.order_polynomial())

    print("Number of linear extensions:")
    print(len(list(P.linear_extensions())))
```

```
Antichain of 5 elements
```


```
Order polynomial:
q^5
Number of linear extensions:
120

Chain of 5 elements
```


```
Order polynomial:
1/120*q^5 + 1/12*q^4 + 7/24*q^3 + 5/12*q^2 + 1/5*q
Number of linear extensions:
1

Product of chains C3 x C5
```




```
Order polynomial:
1/217728000*q^15 + 1/4838400*q^14 + 37/8709120*q^13 + 767/14515200*q^12 +
96667/217728000*q^11 + 7733/2903040*q^10 + 102407/8709120*q^9 +
62509/1612800*q^8 + 2606011/27216000*q^7 + 159871/907200*q^6 + 16159/68040*q^5 +
205847/907200*q^4 + 6091/42000*q^3 + 349/6300*q^2 + 1/105*q
Number of linear extensions:
6006
```

1.6 Exercise 4: Compute the order polynomial of the product of two chains of a and b elements for more (small) values of a and b. Factor these polynomials. Can you conjecture a formula for these polynomials?

1.7 Similarly, compute the number of linear extensions of the product of two chains of a and b elements for more (small) values of a and b. Factor these numbers. Can you conjecture a counting formula?

[78]:
```
for a in range(1,5):
    for b in range(1,3):

```

```
        P = Posets.ChainPoset(a).product(Posets.ChainPoset(b))

        print(f"\nC_{a} x C_{b}")

        op = P.order_polynomial()
        print("order polynomial =", op)
        print("factored =", factor(op))

        le = len(list(P.linear_extensions()))
        print("linear extensions =", le)
        print("factored =", factor(le))
```

```
C_1 x C_1
order polynomial = q
factored = q
linear extensions = 1
factored = 1

C_1 x C_2
order polynomial = 1/2*q^2 + 1/2*q
factored = (1/2) * q * (q + 1)
linear extensions = 1
factored = 1

C_2 x C_1
order polynomial = 1/2*q^2 + 1/2*q
factored = (1/2) * q * (q + 1)
linear extensions = 1
factored = 1

C_2 x C_2
order polynomial = 1/12*q^4 + 1/3*q^3 + 5/12*q^2 + 1/6*q
factored = (1/12) * q * (q + 2) * (q + 1)^2
linear extensions = 2
factored = 2

C_3 x C_1
order polynomial = 1/6*q^3 + 1/2*q^2 + 1/3*q
factored = (1/6) * q * (q + 1) * (q + 2)
linear extensions = 1
factored = 1

C_3 x C_2
order polynomial = 1/144*q^6 + 1/16*q^5 + 31/144*q^4 + 17/48*q^3 + 5/18*q^2 +
1/12*q
factored = (1/144) * q * (q + 3) * (q + 1)^2 * (q + 2)^2
```

```
linear extensions = 5
factored = 5

C_4 x C_1
order polynomial = 1/24*q^4 + 1/4*q^3 + 11/24*q^2 + 1/4*q
factored = (1/24) * q * (q + 1) * (q + 2) * (q + 3)
linear extensions = 1
factored = 1

C_4 x C_2
order polynomial = 1/2880*q^8 + 1/180*q^7 + 53/1440*q^6 + 47/360*q^5 +
769/2880*q^4 + 113/360*q^3 + 47/240*q^2 + 1/20*q
factored = (1/2880) * q * (q + 4) * (q + 1)^2 * (q + 2)^2 * (q + 3)^2
linear extensions = 14
factored = 2 * 7
```

I wasn't able to really conjecture but the formulas are: orderpoly_{C_a x C_b}(n) = *{i=0}^{a-1}* {j=0}^{b-1} (q+i+j)/(i+j+1) and # linext = (ab)! / *{i=0}^{a-1}* {j=0}^{b-1}(i+j+1).